BASE DE DATOS

2026



Ing. Esteban Schab
AUS. Gabriel Casares
Aux. Alumno Tomas Viñuela

INTEGRIDAD DE LOS DATOS

Consistencia, Transacciones, Recuperación y Concurrencia

INTEGRIDAD

Mantenerse entero, completo y fiel a sus principios o definiciones.

CONSISTENCIA

Cualidad de lo que es estable, sólido, coherente, o que mantiene una regularidad en el tiempo.

RECUPERACIÓN

La **recuperación** en un sistema de base de datos consiste básicamente en el reestablecimiento de la base de datos a un estado **consistente**.

FALLAS

Una Base de Datos se puede encontrar en estado inconsistente por diferentes motivos:

- Falla en el sistema
- Corte de luz
- Problemas de hardware (fallas de memoria o disco rígido)
- Error de diseño / implementación
- Concurrencia

EJ. PROBLEMA POR CONCURRENCIA



T1	T2
read_item(x)	
x:= x – n	
	read_item(y)
	x:= x + m
write_item(x)	
read_item(x)	
	write_item(x)
y:= y + m	
write_item(y)	

TRANSACCIÓN

Una **transacción** es una unidad lógica de procesamiento de una Base de Datos, compuesta por una **secuencia de una o más operaciones** de acceso (inserción, modificación, borrado, consulta), cuyo resultado final es que se ejecuten todas o ninguna de ellas.

VENTAJA

- Si una operación falla sin una transacción, se corre el riesgo de dejar los datos inconsistentes. Por ejemplo: dinero descontado, pero no abonado.
- Al agrupar las operaciones en una transacción, se asegura que todas las operaciones se completen o no se aplique ninguna.

PROPIEDADES ACID

Propiedad	Descripción
Atomicidad	La atomicidad implica que una transacción es todo o nada. Si alguna parte falla, se revierte por completo y la base de datos queda como estaba.
Consistencia	La consistencia asegura que una transacción lleva la base de datos de un estado válido a otro. Es decir, todas las reglas, restricciones y relaciones se respetan durante toda la transacción.
Aislamiento	El aislamiento garantiza que las transacciones no entren en conflicto entre sí, incluso cuando se ejecutan simultáneamente. Si dos usuarios actualizan el mismo registro, el aislamiento evita que los cambios de uno sobrescriban o corrompan los del otro. Los niveles de aislamiento, como READ COMMITTED y SERIALIZABLE, determinan lo estricta que es esta separación y equilibran rendimiento y consistencia.
Durabilidad	La durabilidad garantiza que los cambios de una base de datos son permanentes una vez que se confirma la transacción, incluso ante un fallo del sistema.

COMANDOS UTILIZADOS

- **Begin**:
  Marca el inicio de una transacción. Todas las operaciones posteriores formarán parte de ella.
- **Commit**:
  Finaliza la transacción, haciendo permanentes todos los cambios en la base de datos.
- **Rollback**:
  Deshace todos los cambios realizados durante la transacción, devolviendo la base de datos a su estado anterior en caso de error o fallo.

COMANDOS UTILIZADOS

- **Savepoint <__>**:
  Generar un punto de recuperación.
- **Rollback to <__>**:
  Volver a un punto de recuperación.
- **Release savepoint**:
  Libera los recursos del punto de recuperación.

CONCURRENCIA

- Los DBMS permiten que varios usuarios se conecten a la BB.DD. al mismo momento y que realicen diversas transacciones al mismo tiempo.
- Se deben administrar estas transacciones para mantener en todo momento la consistencia de los datos.

MECANISMOS DE BLOQUEO

- Permiten controlar el acceso a los recursos y aseguran la integridad transaccional.
- Los bloqueos compartidos permiten que varias transacciones lean un recurso evitando modificaciones y manteniendo la consistencia durante la lectura.
- Por su parte, los bloqueos exclusivos restringen el acceso del resto para garantizar escritura exclusiva.

```
SELECT * FROM inventory WITH (ROWLOCK, HOLDLOCK) WHERE product_id = 101;
```

NIVELES DE AISLAMIENTO

Los niveles de aislamiento determinan cómo interactúan las transacciones entre sí y equilibran el rendimiento con la consistencia de los datos.

NIVELES DE AISLAMIENTO

Nivel de aislamiento	Lectura sucia	Lectura no repetible	Lectura fantasma	Anomalía de serialización
Read uncommitted	Permitido, pero no en PG	Posible	Posible	Posible
Read committed	No es posible	Posible	Posible	Posible
Repeatable read	No es posible	No es posible	Permitido, pero no en PG	Posible
Serializable	No es posible	No es posible	No es posible	No es posible

https://www.postgresql.org/docs/current/transaction-iso.html

NIVELES DE AISLAMIENTO

- **Read Uncommitted:** permite lecturas sucias (leer datos no confirmados), mejorando el rendimiento al minimizar la sobrecarga de bloqueos.
- **Read Committed:** Evita lecturas sucias. Predeterminado en la mayoría de los DBMS (como PostgreSQL y SQL Server).

NIVELES DE AISLAMIENTO

- **Repeatable read:** En Postresql no permite lectura fantasma (aparición de nuevas filas en consultas de rango).
  A comparación con Read Committed, cada consulta ve una instantánea del estado al inicio de la primera instrucción sin control de transacción, no al inicio de la instrucción actual.
- **Serializable:** garantiza el nivel más alto de consistencia al aislar completamente las transacciones, aunque puede reducir la concurrencia.

USO DE SAVEPOINTS PARA REVERSIONES PARCIALES

Los savepoints permiten crear puntos de control dentro de una transacción.

Si se produce un error, se puede deshacer hasta un savepoint concreto en lugar de revertir todo.

```
-- Start Transaction
BEGIN TRANSACTION;
-- Savepoint for first operation
SAVEPOINT step1;
-- First Operation: Debit Account 1
UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
-- Optional: Rollback to step1 if needed
-- ROLLBACK TO step1;
-- Second Operation: Credit Account 2
UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
-- Commit Transaction
COMMIT;
```

IMPLEMENTAR MECANISMOS DE MANEJO DE ERRORES

Un manejo de errores eficaz asegura que las transacciones se completen correctamente o fallen de forma controlada.

Usar Bloques TRY CATCH para gestionan errores dinámicamente dentro de una transacción.

El DBMS mantiene un registro de transacciones mediante logs para rastrear errores y estados.

BLOQUES TRY CATCH

```
-- Example of error handling with TRY CATCH
BEGIN TRY
    BEGIN TRANSACTION;
    UPDATE accounts SET balance = balance - 100 WHERE account_id = 1;
    UPDATE accounts SET balance = balance + 100 WHERE account_id = 2;
    COMMIT;
END TRY
BEGIN CATCH
    ROLLBACK;
    PRINT 'Transaction failed and rolled back.';
END CATCH;
```

BIBLIOGRAFÍA

- C. J. Date, "Introducción a los Sistemas de Bases de Datos" séptima edición, Prentice Hall (2001) ISBN 968-444-419-2

- Ramez A. Elmasri, Shamkant B. Navathe, “Fundamentos de Sistemas de Bases de Datos”, Addison Wesley – Pearson (2003) ISBN 978-8478290512

- Reinosa Enrique J., Muñoz Roberto, et. al., "Bases de Datos", 1a ed., Buenos Aires, Alfaomega Grupo Editor Argentino (2012), ISBN 978-987-1609-31-4.

FUENTES

- https://www.datacamp.com/es/tutorial/sql-transactions
- https://www.postgresql.org/docs/current/transaction-iso.html
- https://medium.com/@rahulhind/the-core-of-postgresql-understanding-transactions-isolation-and-mvcc-90247992f14e
- https://www.pythian.com/blog/monitoring-transaction-logs-in-postgresql